

AI Lab Task 5

Name : Mehr Ali BSCS-4B

A* Search Algorithm for Romania Map

This lab implements the A* search algorithm to find the shortest path between two cities on the Romania map.

The heuristic function used is the straight-line distance between cities based on their latitude and longitude.

```
In [13]: lat_long = {
    'Arad': (46.1667, 21.3167),
    'Bucharest': (44.4167, 26.1000),
    'Craiova': (44.3333, 23.8167),
    'Drobeta': (44.6259, 22.6566),
    'Eforie': (44.0667, 28.6333),
    'Fagaras': (45.8416, 24.9730),
    'Giurgiu': (43.9037, 25.9699),
    'Hirsova': (44.6833, 27.9500),
    'Iasi': (47.1585, 27.6014),
    'Lugoj': (45.6904, 21.9033),
    'Neamt': (46.9283, 26.3705),
    'Oradea': (47.0553, 21.9214),
    'Pitesti': (44.8565, 24.8697),
    'Rimnicu Vilcea': (45.1042, 24.3758),
    'Sibiu': (45.7977, 24.1512),
    'Timisoara': (45.7489, 21.2087),
    'Urziceni': (44.7167, 26.6333),
    'Vaslui': (46.6333, 27.7333),
    'Zerind': (46.6225, 21.5174)
}

In [14]: graph = {
    'Arad': {'Zerind': 75, 'Sibiu': 140, 'Timisoara': 118},
    'Zerind': {'Arad': 75, 'Oradea': 71},
    'Oradea': {'Zerind': 71, 'Sibiu': 151},
    'Sibiu': {'Arad': 140, 'Oradea': 151, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
    'Timisoara': {'Arad': 118, 'Lugoj': 111},
    'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
    'Mehadia': {'Lugoj': 70, 'Drobeta': 75},
    'Drobeta': {'Mehadia': 75, 'Craiova': 120},
    'Craiova': {'Drobeta': 120, 'Rimnicu Vilcea': 146, 'Pitesti': 138},
    'Rimnicu Vilcea': {'Sibiu': 80, 'Craiova': 146, 'Pitesti': 97},
    'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
    'Pitesti': {'Rimnicu Vilcea': 97, 'Craiova': 138, 'Bucharest': 101},
    'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90, 'Urziceni': 85},
    'Giurgiu': {'Bucharest': 90},
```

```
'Urziceni': {'Bucharest':85,'Hirsova':98,'Vaslui':142},
'Hirsova': {'Urziceni':98,'Eforie':86},
'Eforie': {'Hirsova':86},
'Vaslui': {'Urziceni':142,'Iasi':92},
'Iasi': {'Vaslui':92,'Neamt':87},
'Neamt': {'Iasi':87}
}
```

In [15]: **class** Node:

```
    def __init__(self, city, parent=None, g=0, h=0):
        self.city = city
        self.parent = parent
        self.g = g
        self.h = h
        self.f = g + h

    def __lt__(self, other):
        return self.f < other.f
```

In [16]: **import** math

```
def heuristic(city1, city2):

    lat1, lon1 = lat_long[city1]
    lat2, lon2 = lat_long[city2]

    return math.sqrt((lat1-lat2)**2 + (lon1-lon2)**2)*111
```

In [17]: **import** heapq

```
def astar(start, goal):

    open_list=[]
    closed=set()

    start_node=Node(start,None,0,heuristic(start,goal))
    heapq.heappush(open_list,start_node)

    while open_list:

        current=heapq.heappop(open_list)

        if current.city==goal:

            path=[]
            distance=current.g

            while current:
                path.append(current.city)
                current=current.parent

            return path[::-1],distance

        closed.add(current.city)
```

```
for neighbor, cost in graph[current.city].items():  
  
    if neighbor in closed:  
        continue  
  
    g = current.g + cost  
    h = heuristic(neighbor, goal)  
  
    neighbor_node = Node(neighbor, current, g, h)  
  
    heapq.heappush(open_list, neighbor_node)
```

```
In [18]: path, distance = astar("Arad", "Bucharest")  
  
print("Shortest path from Arad to Bucharest:", path)  
print("Distance:", round(distance))
```

Shortest path from Arad to Bucharest: ['Arad', 'Sibiu', 'Fagaras', 'Bucharest']
Distance: 450

Conclusion

The A* search algorithm successfully finds the shortest path between Arad and Bucharest.

Shortest path: Arad → Sibiu → Fagaras → Bucharest

Total distance = 450 km